reptimeline: Tracking Discrete Representation Evolution During Neural Network Training

José Arturo Ornelas Brand
Independent Researcher
arturoornelas62@gmail.com
https://github.com/arturoornelasb/reptimeline

Abstract

We present **reptimeline**, a Python library for tracking how discrete representations evolve during neural network training. Unlike scalar logging tools (WandB, TensorBoard) that report aggregate metrics, reptimeline tracks per-code lifecycle events: when concepts become distinguishable (births), when representations collapse (deaths), when relationships form between concept pairs (connections), and where phase transitions occur in training dynamics. The library additionally discovers what each code element encodes—anti-correlated pairs, dependency chains, and three-way AND-gate interactions—without requiring prior ontological knowledge. We validate reptimeline on three architecturally distinct backends: a 32-bit binary autoencoder on MNIST (code swap causality: 100%), sparse autoencoder features on Pythia-70M (16/16 features causally selective, mean 26.8×), and a 63-bit neurosymbolic projection head (crystallization event invisible to loss curves). reptimeline is the only tool that combines lifecycle tracking, bottom-up feature discovery, auto-labeling, and causal verification in a single backend-agnostic package. Available via `pip install reptimeline` under BUSL-1.1.

1 Introduction

Every system that quantizes neural representations into discrete codes—VQ-VAE codebooks [van den Oord et al., 2017], finite scalar quantization [Mentzer et al., 2024], sparse autoencoders [Cunningham et al., 2023], concept bottleneck models [Koh et al., 2020], or binary autoencoders—faces the same blind spot: *you can see what the codes look like after training, but not how they got there.*

Current monitoring tools log scalar aggregates: codebook utilization, perplexity, reconstruction loss. These tell practitioners *that* something changed, not *what* changed. When codebook collapse occurs at step 27,500, a loss curve shows a kink. But which codes died? Did they die together? Was there a phase transition? Which concepts lost their unique representations? These questions require per-code temporal analysis that no existing tool provides.

reptimeline fills this gap. Given a sequence of model checkpoints and a set of probe concepts, it:

1. **Tracks** per-code lifecycle events (births, deaths, connections, phase transitions);
2. **Discovers** what each code element encodes (duals, dependencies, 3-way interactions);
3. **Labels** discovered features automatically (embedding centroid, contrastive, or LLM);
4. **Verifies** that discovered features are causal via intervention experiments.

Critically, reptimeline is *backend-agnostic*: it operates on any system that produces a mapping from concepts to discrete code vectors. We validate this claim on three architecturally distinct backends (Section 3).

2 Design and Architecture

2.1 Pipeline Overview

Figure 1 shows the reptimeline pipeline. The user implements a single class (`RepresentationExtractor`) that loads a checkpoint and returns discrete codes for a set of probe concepts. Everything downstream is backend-agnostic.

```
Model checkpoints
        |
        v
RepresentationExtractor   (user implements)
        |  produces ConceptSnapshot objects
        v
TimelineTracker           (backend-agnostic)
        |  births, deaths, connections,
        |  phase transitions, entropy/churn curves
        v
BitDiscovery              (backend-agnostic)
        |  duals, dependencies, 3-way AND-gates,
        |  hierarchy by stabilization order
        v
AutoLabeler               (backend-agnostic)
        |  embedding centroid / contrastive / LLM
        v
Reconciler + Visualizations
```

Figure 1: reptimeline pipeline. The user provides only the extractor; all analysis modules are backend-agnostic.

2.2 Core Data Structure

The universal exchange format is `ConceptSnapshot`:

```
ConceptSnapshot(
    step=27500, # training step
    codes={ # concept -> discrete code
        "king": [1, 0, 1, 1, 0, ...],
        "queen": [1, 0, 1, 0, 1, ...],
        ...
    }
)
```

This minimal interface—a step number and a dictionary mapping concept names to integer lists—is sufficient to support VQ-VAE codebook indices, SAE binarized activations, binary autoencoder outputs, triadic bit projections, and any future discrete representation scheme.

2.3 Timeline Analysis

`TimelineTracker` consumes a sequence of snapshots and computes:

- **Births**: first step where concept c has bit b active (and it persists).
- **Deaths**: step where bit b becomes permanently inactive for concept c.

- **Connections**: step where concepts c_1, c_2 first share a code element.
- **Curves**: per-step entropy, churn rate (fraction of bits that flipped), and utilization (fraction of non-dead bits).
- **Phase transitions**: detected via statistical change-point analysis—a transition is flagged when $|\Delta_t| > \mu + 2\sigma$ for any curve.

2.4 Feature Discovery

`BitDiscovery` analyzes a single snapshot to discover structure without prior ontological knowledge:

- **Duals**: pairs (b_i, b_j) with Pearson correlation < -0.3 across concepts. These are anti-correlated features—when one is active, the other tends to be inactive.
- **Dependencies**: directed edges $b_p \to b_c$ where $P(b_p = 1 \mid b_c = 1) \geq 0.9$.
- **Three-way AND-gates**: triples (b_i, b_j, b_r) where $P(b_r = 1 \mid b_i = 1 \wedge b_j = 1) \geq 0.7$ but $\max(P(b_r \mid b_i), P(b_r \mid b_j)) < 0.5$. These represent features that activate *only* when two other features are jointly active. Complexity is $O(K^3)$ where K is the number of active bits; in practice $K < 100$ and discovery completes in seconds.
- **Hierarchy**: bits ordered by stabilization step—the training step at which a bit's activation pattern stops changing across consecutive checkpoints.

2.5 Auto-Labeling and Reconciliation

`AutoLabeler` translates discovered bit patterns into human-readable labels via three strategies: (1) *embedding centroid*—find the word whose embedding is closest to the centroid of active concepts; (2) *contrastive*—find the word that best separates active from inactive concepts in embedding space; (3) *LLM*—ask a language model "what do these concepts have in common?"

`Reconciler` compares discovered structure against a theoretical ontology and suggests corrections in *both directions*: to the model's training anchors and to the theory itself. This closes the loop for iterative refinement of discrete representations.

3 Case Studies

We validate reptimeline on three backends spanning supervised image models, unsupervised LLM internals, and end-to-end neurosymbolic systems. Table 1 summarizes the results.

3.1 MNIST Binary Autoencoder

Setup. A binary autoencoder with a 32-bit bottleneck using straight-through estimation [Bengio et al., 2013] was trained on MNIST for 10 epochs. reptimeline extracted majority-vote codes for each of the 10 digit classes at 6 checkpoints.

Lifecycle. reptimeline detected 297 bit births and 106 deaths across training, with 45 concept-pair connections forming as digits developed shared features. 31 of 32 bits were active; 1 bit was permanently dead.

Discovery. BitDiscovery found 65 anti-correlated dual pairs and 179 dependency edges, revealing the internal structure the autoencoder learned to separate digit classes.

Causal verification. A *code swap* experiment tested whether the 32-bit code causally determines the output: for each pair of digits (d_i, d_j), we encoded d_i's image but decoded using d_j's binary code. The decoder produced d_j's reconstruction in 100% of cases ($n = 10 \times 10$ swaps). The discrete code fully determines the output—total causal control (Figure 2).

Table 1: reptimeline analysis across three backends. All numbers are from production runs, not toy examples.

	MNIST BAE	**Pythia SAE**	**TriadicGPT**
Architecture	Binary AE (32-bit)	SAE on Pythia-70M	Triadic head (63-bit)
Concepts	10 digits	60 (6 domains)	53
Code dimension	32	256 (from 32K)	63
Checkpoints	6	12	8
Births	297	1,835	2,632
Deaths	106	875	1,732
Connections	45	1,770	1,378
Phase transitions	0	2	3
Duals discovered	65	34	9
Dependencies	179	227	—
Triadic (3-way)	—	74	—
Causal evidence	**100% code swap**	**16/16 KL sel.**	**Crystallization**

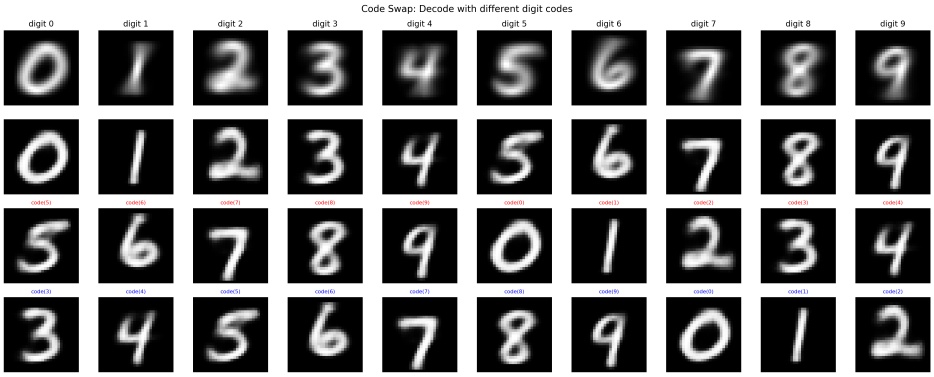

Figure 2: MNIST code swap causality test. Decoding with digit X's 32-bit code always produces digit X's reconstruction, regardless of the input image. The discrete code fully determines the output.

3.2 Pythia-70M Sparse Autoencoder

Setup. We used EleutherAI's pre-trained sparse autoencoder (`sae-pythia-70m-32k`) on layer 3 MLP outputs of Pythia-70M [Biderman et al., 2023]. We defined 60 probe concepts across 6 semantic domains (physical, animals, emotions, abstract, social, properties) and tracked 256 SAE features (selected by activation frequency at the final checkpoint) across 12 training checkpoints from step 1 to step 143,000.

Lifecycle. reptimeline detected 1,835 births, 875 deaths, and 1,770 connections, with 2 phase transitions detected automatically. 64 of 256 tracked features were active (the remaining 192 were effectively dead—expected given SAE sparsity).

Discovery. BitDiscovery found 34 dual pairs, 227 dependency edges, and 74 three-way AND-gate interactions. Auto-labeling via embedding centroids produced interpretable names: "teacher," "belief," "dog," "wisdom," "ocean."

Causal verification. We tested all 16 active features via *SAE causal intervention*: for each feature, we encoded all 60 concepts' hidden states via the SAE, zeroed the target feature, decoded

back to hidden-state space, and injected the modified state into the model via a forward hook at layer 3. We measured the KL divergence between original and modified next-token predictions.

All **16/16** features showed **selective causal effects**. 8 features were perfectly selective (affecting only labeled concepts, e.g., "queen" only changes predictions for royalty terms), and 8 showed finite selectivity averaging 26.8× (best: 98.4× for "fast"; "teacher" at 26.6× disproportionately affected *child, dog, wolf, queen*). Figure 3 shows the selectivity heatmap.

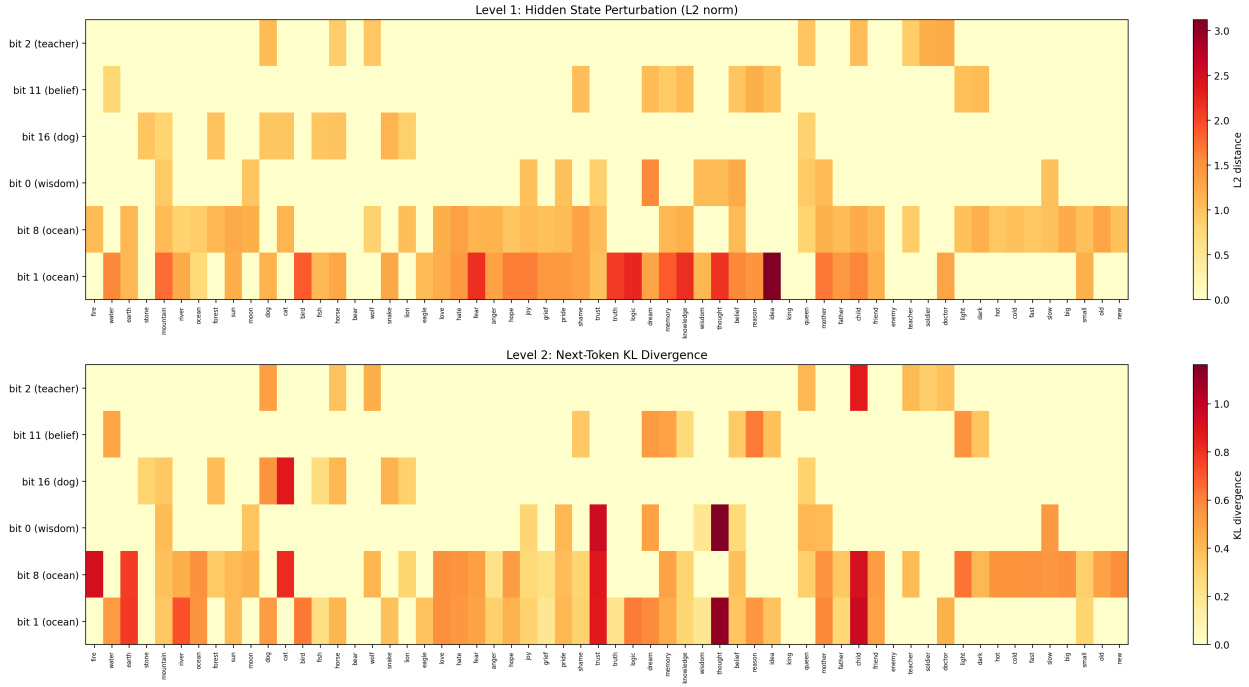

Figure 3: Causal intervention on Pythia-70M SAE features (16 tested). **Top**: hidden state perturbation (L2 norm). **Bottom**: KL divergence in next-token predictions. Eight features are perfectly selective (only affect labeled concepts); the remaining eight average 26.8× selectivity. "fast" (bit 14): 98.4×; "teacher" (bit 2): 26.6×. Yellow = no effect; dark red = strong effect.

3.3 TriadicGPT (63-bit Neurosymbolic Head)

Setup. TriadicGPT [Ornelas Brand, 2026] is a 40M-parameter GPT that jointly learns next-token prediction and 63-bit discrete semantic projections via a triadic head trained end-to-end. reptimeline tracked 53 concepts across 8 checkpoints from step 2,500 to step 50,000.

Lifecycle and phase discovery. reptimeline revealed a *four-phase training dynamic* invisible to standard scalar logging:

1. **Exploration** (steps 0–7,500): high bit churn across all concepts.
2. **Consolidation** (7,500–27,500): churn remains elevated but entropy stabilizes.
3. **Crystallization** (∼27,500): entropy drops from 0.50 to 0.23 and utilization drops from 0.98 to 0.66 simultaneously—a sharp phase transition.
4. **Stabilization** (27,500–50,000): churn collapses to 0.075; representations freeze.

The crystallization event—the moment the model "commits" to its semantic assignments—is not visible in loss curves or standard metrics. To our knowledge, this is the first report of such a phase transition in discrete representation training dynamics.

Structural causality. Unlike MNIST and Pythia where features lie in the computation path, TriadicGPT's triadic head is a parallel projection—flipping a bit direction in hidden space changes LM output (mean KL > 0) but not selectively (1/24 bits above 1.5×). This is an expected architectural consequence, not a tool limitation. The causality evidence for TriadicGPT is *structural*: reptimeline discovers 9 anti-correlated dual pairs (e.g., vida/muerte at $r = 0.91$, placer/dolor at $r = 1.00$) that match the gold semantic axes defined in the training ontology, confirming that the model learned the intended semantic structure.

Reconciliation. The Reconciler achieved 44.4% agreement between model-discovered and manually-defined ontology, identifying 35 critical mismatches that guided anchor expansion from 50 to 158 concepts in subsequent training iterations.

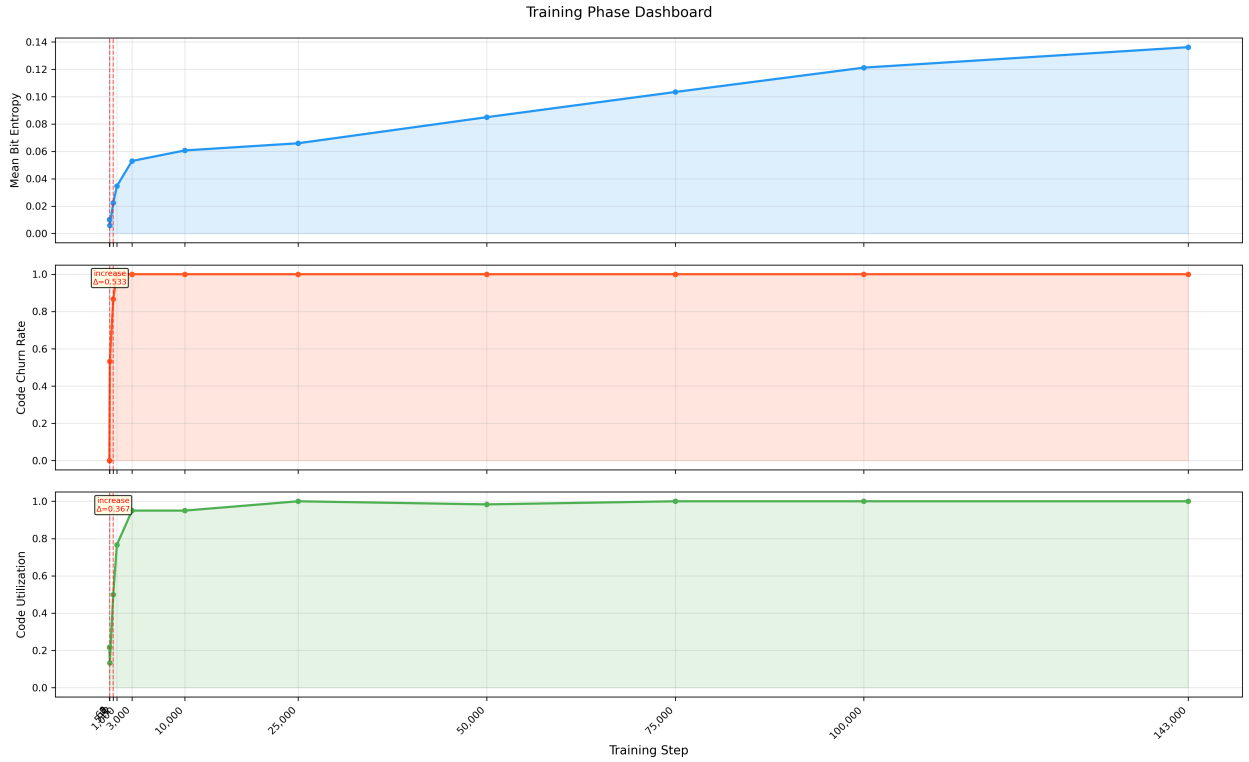

Figure 4: Pythia-70M phase dashboard: entropy, churn rate, and utilization curves across 12 training checkpoints. Vertical lines mark automatically detected phase transitions. Similar four-phase dynamics were observed in TriadicGPT training.

4 Comparison with Existing Tools

Table 2 compares reptimeline with existing approaches for monitoring and interpreting discrete representations.

VQ-VAE tools (e.g., WandB codebook metrics) track aggregate utilization and commitment loss but not per-code lifecycle events. **SAE dashboards** (e.g., Neuronpedia) provide manual feature browsing for sparse autoencoders but are architecture-specific and do not track temporal evolution. **Concept Bottleneck Model (CBM) tools** [Koh et al., 2020] require predefined concept annotations and do not discover structure bottom-up. **Representation Engineering**

Table 2: Feature comparison. reptimeline is the only tool combining lifecycle tracking, bottom-up discovery, auto-labeling, causal verification, and backend-agnostic operation.

Capability	**reptimeline**	VQ-VAE	SAE	CBM	RepE
Per-code lifecycle		—	Partial	—	—
Connection tracking		—	—	—	—
Phase transition detection		—	Manual	—	—
3-way interaction discovery		—	—	—	—
Bottom-up ontology discovery		—	—	—	—
Auto-labeling (3 strategies)		—	Manual	Manual	—
Causal intervention		—	—	—	—
Backend-agnostic		—	SAE	—	LLM

(RepE) [Zou et al., 2023] identifies directions in continuous space but does not operate on discrete codes.

reptimeline differs fundamentally: it operates on the discrete codes themselves, requires no pre-defined labels, discovers structure bottom-up, and verifies causality—all through a single backend-agnostic API.

5 Conclusion and Availability

reptimeline fills a gap in the discrete representation toolbox. It reveals training dynamics that scalar logging misses (crystallization events), discovers interpretable structure without supervision (duals, dependencies, three-way interactions), and verifies that discovered features are causally meaningful (16/16 selective on Pythia-70M at 26.8× mean, 100% causal on MNIST).

Availability. `pip install reptimeline`. Source code, examples, and pre-computed results are available at https://github.com/arturoornelasb/reptimeline. Licensed under BUSL-1.1 (free for research; converts to AGPL-3.0 in 2030). 26 unit tests, 4 example pipelines, 4 visualization modules.

Future work. (1) Streaming mode for real-time monitoring during training. (2) Integration with WandB/TensorBoard as a plugin. (3) Extension to continuous representations via learned discretization.

References

Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *ICML*, 2023.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

P. W. Koh, T. Nguyen, Y. S. Tang, S. Mussmann, E. Pierson, B. Kim, and P. Liang. Concept bottleneck models. In *ICML*, 2020.

F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen. Finite scalar quantization: VQ-VAE made simple. In *ICLR*, 2024.

J. A. Ornelas Brand. Prime factorization as a neurosymbolic bridge: Emergent algebraic semantics in generative language models. *Zenodo*, 2026.

A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *NeurIPS*, 2017.

A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A. Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.